What is a context window?

Machine learning
Welcome
- Introduction
- Data science for machine learning
- Feature Engineering

eBook
How to choose the right foundation model
Learn how to choose the right

By Dave Bergmann

What is a context window?

The context window (or “context length”) of a large language model (LLM) is the amount of text, in tokens, that the model can consider or “remember” at any one time. A larger context window enables an AI model to process longer inputs and incorporate a greater amount of information into each output.

An LLM’s context window can be thought of as the equivalent of its working memory. It determines how long of a conversation it can carry out without forgetting details from earlier in the exchange. It also determines the maximum size of documents or code samples that it can process at once. When a prompt, conversation, document or code base exceeds an artificial intelligence model’s context window, it must be truncated or summarized for the model to proceed.

Generally speaking, increasing an LLM’s context window size translates to increased accuracy, fewer hallucinations, more coherent model responses, longer conversations and an improved ability to analyze longer sequences of data. However, increasing context length is not without tradeoffs: it often entails increased computational power requirements—and therefore increased costs—and a potential increase in vulnerability to adversarial attacks.

The latest AI trends, brought to you by experts
Get curated insights on the most important—and intriguing—AI news. Subscribe to our weekly Think newsletter. See the IBM Privacy Statement.
First name*
Last name*
Business email*
* All fields required
Your subscription will be delivered in English. You will find an unsubscribe link in every newsletter. Refer to our IBM Privacy Statement for more information.
Subscribe

Context windows and tokenization

In real-world terms, the context length of a language model is measured not in words, but in *tokens.* To understand how context windows work in practice, it’s important to understand how these tokens work.

The way LLMs process language is fundamentally different from the way humans do. Whereas the smallest unit of information we use to represent language is a single *character*—such as a letter, number or punctuation mark—the smallest unit of language that AI models use is a token. To train a model to understand language, each token is assigned an ID number; these ID numbers, rather than the words or even the tokens themselves, are used to train the model. This *tokenization* of language significantly reduces the computational power needed to process and learn from the text.

There is a wide variance in the amount of text that one token can represent: a token can stand in for a single character, a part of a word (such as a suffix or prefix), a whole word or even a short multiword phrase. Consider the different roles played by the letter “*a* ” in the following examples:

```
“Jeff drove a car.”
```

Here, “*a*“ is an entire word. In this situation, it would be represented by a distinct token.

```
“Jeff is amoral.”
```

Here, “*a*“ is not a word, but its addition to *moral* significantly changes the meaning of the word. *Amoral* would therefore be represented by two distinct tokens: a token for *a* and another for *moral.*

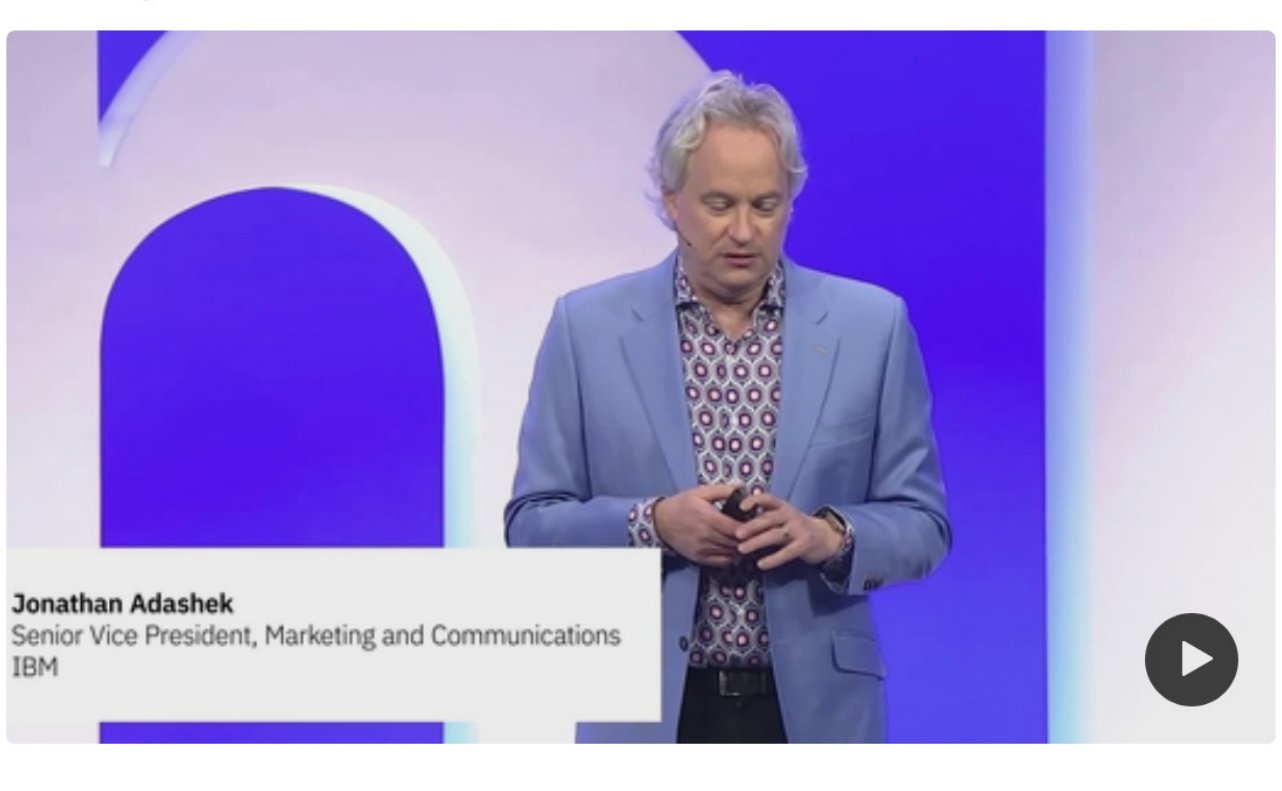
```
"Jeff loves his cat."
```

Here, *a* is simply a letter in the word “*cat*.” It carries no semantic meaning unto itself and would, therefore, *not* need to be represented by a distinct token.

There is no fixed word-to-token “exchange rate,” and different models or tokenizers—a modular subset of a larger model responsible for tokenization—might tokenize the same passage of writing differently. Efficient tokenization can help increase the actual amount of text that fits within the confines of a context window. But for general purposes, a decent estimate would be roughly 1.5 tokens per word. The Tokenizer Playground on Hugging Face is an easy way to see and experiment with how different models tokenize text inputs.

Variations in linguistic structure and representation in training data can result in some languages being more efficiently tokenized than others. For example, an October 2024 study explored an example of the same sentence being tokenized in both English and Telugu. Despite the Telugu translation having significantly fewer characters than its English equivalent, it resulted in over 7 times the number of tokens in context.

Think Keynotes


Jonathan Adashek
Senior Vice President, Marketing and Communications
IBM

How enterprises excel in the AI era

Move beyond AI hype to measurable value. See how IBM is transforming into an AI-first enterprise and turning agentic AI into productivity, reinvestment and real business impact.

Build with watsonx Orchestrate® →

Why do models have a maximum context length?

Though context windows are usually associated with LLMs used for summarization, generating text and other natural language processing (NLP) tasks, context length as a technical consideration is not exclusive to language models. The notion of a context window is relevant to *any machine learning model that uses the transformer architecture,* which comprises most modern generative AI models, including nearly all LLMs.

Transformer models use a *self-attention* mechanism to calculate the relationships and dependencies between different parts of an input (like words at the beginning and end of a paragraph). Mathematically speaking, a self-attention mechanism computes vectors of weights for each token in a sequence of text, in which each weight represents how relevant that token is to others in the sequence. An autoregressive LLM iteratively consults those weights each time it generates the next word of its output. The size of the context window determines the maximum number of tokens that the model can “pay attention to” at any one time.

It's worth noting that the text of the actual user input is often not the only thing taking up space within a model’s context window. In many cases, such as with chatbots, models are also provided with a “system prompt”—often hidden from the user—that conditions their behavior and governs other aspects of the conversation. Supplementary information drawn from external data sources for retrieval augmented generation (RAG) is likewise stored within the context window during inference. Special characters, line breaks and other formatting measures also consume some portion of the available context.

It’s also worth noting that language models are not the only neural networks that utilize transformers. For instance, some diffusion models used for image generation incorporate self-attention into their architecture. In their case, the context being attended to is not between tokens representing words (or parts of words) in written content, but between *pixels* in an image. In such a model, context length would apply to the number of pixels whose relationships the model must understand. Its context window could be exceeded by a high-resolution image containing too many pixels to process at once.

Context windows and computing resources

Equipping a model with a large context window comes at a cost, both figuratively and literally. Compute requirements scale quadratically with the length of a sequence: for instance, if the number of input tokens doubles, the model needs 4 times as much processing power to handle it.

Similarly, increasing context length can also slow down outputs. Each time the model autoregressively predicts the next token in a sequence, it computes the relationships between that token and every single preceding token in the sequence. Inference might be relatively fast at the beginning of a sequence or conversation but progressively become slower as the context length increases. This is problematic for use cases requiring near-instantaneous inference in real-time.

Recent advancements in average context length for language models have been partially enabled by new techniques to increase inference speed and efficiency enough to sufficiently offset these inherent tradeoffs. These optimization techniques have allowed even small, open source modern LLMs to offer context windows exponentially larger than that of the original GPT-3.5 model that launched OpenAI’s ChatGPT in late 2022.

Challenges of long context windows

Even when adequate measures are taken to offset the tradeoffs in computation requirements and processing speed, extending a model’s context length limit introduces additional challenges and complications.

Performance challenges

Like people, LLMs can be overwhelmed by an abundance of extra detail. They can also get lazy and take cognitive shortcuts. A 2023 paper found that LLMs don’t “robustly make use of information in long input contexts.” More specifically, the authors observed that models perform best when relevant information is toward the beginning or end of the input context. They further observed that performance *degrades* when the model must carefully consider the information in the *middle* of long contexts.[1]

Novel methods to improve the efficacy of the transformer’s self-attention mechanism itself, such as rotary position embedding (RoPE), aim to modify the positional encoding of tokens in attention vectors. The widespread adoption of RoPE-based methods has yielded enhanced performance and speed on tasks involving tokens at a large distance from one another.

Ongoing research has produced a number of benchmarks designed to measure an LLM’s ability to effectively find and use relevant information with large passages, such as needle-in-a-haystack (NIAH), RULER and LongBench.

Safety and cybersecurity challenges

A longer context window might also have the unintended effect of presenting a longer attack surface for adversarial prompts. Recent research from Anthropic demonstrated that increasing a model’s context length also increases its vulnerability to “jailbreaking” and (subsequently) being provoked to produce harmful responses.[2]

Context window sizes of prominent LLMs

The average context window of a large language model has grown exponentially since the original generative pretrained transformers (GPTs) were released. To date, each successive generation of LLMs has typically entailed significantly longer context lengths. At present, the largest context window offered by a prominent commercial model is over 1 million tokens. It remains to be seen whether context windows will continue to expand or if we’re already approaching the upper limit of practical necessity.

For reference, here are the current context lengths offered by some commonly cited models and model families as of October 2024.

OpenAI's GPT series

- The GPT-3.5 model that powered the launch of ChatGPT had a maximum context length of 4,096 tokens, later expanded to 8,192 tokens with GPT-3.5-Turbo.
- At launch, GPT-4 had that same 8,192-token context length. Though the context window of both GPT-4 and GPT-4-Turbo has since been increased to 128,000 tokens, its max output tokens remain capped at 4,096 tokens.
- Both GPT-4o and GPT-4o mini have a context window of 128,000 tokens, with output capped at 16,384 tokens.

The new o1 model family likewise offers a context window of 128,000 tokens, though they offer greater output context length.

Meta Llama models

The original Llama models had a maximum context length of 2,048 tokens, which was doubled to 4,096 tokens for Llama 2. During their launch in April 2024, Llama 3 models offered a context window of roughly 8,000 tokens.

Llama’s context length was significantly increased with the launch of Llama 3.1 models, which offered 128,000-token long context windows. Llama 3.2 models likewise have a maximum context length of 128,000 tokens.

Mistral Large 2

Mistral Large 2, the flagship model offered by Mistral AI, has a context window of 128,000 tokens.

Google Gemini models

Google’s Gemini series of models offers what is currently the largest context window amongst commercial language models. Gemini 1.5 Pro, Google’s flagship model, offers a context length of up to 2 million tokens. Other Gemini 1.5 models, such as Gemini 1.5 Flash, have a context window of 1 million tokens.

Anthropic's Claude models

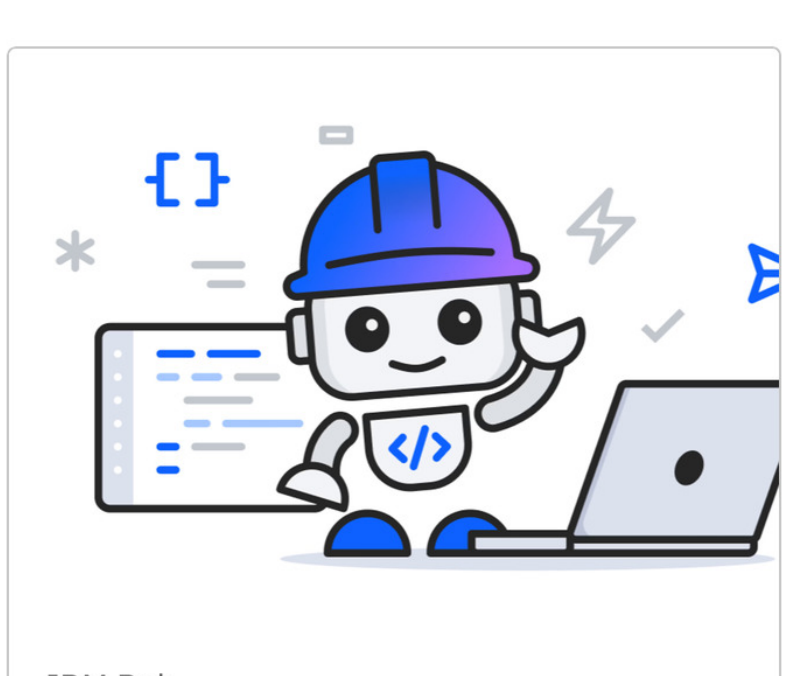
Anthropic’s latest Claude models, such as the Claude 3.5 Sonnet, offer a standard context window of about 200,000 tokens. In early September 2024, Anthropic announced that the models accessed through its new “Claude Enterprise” plan would offer an expanded 500,000 token context window.

Author

Dave Bergmann
Senior Staff Writer, AI Models
IBM Think

Share

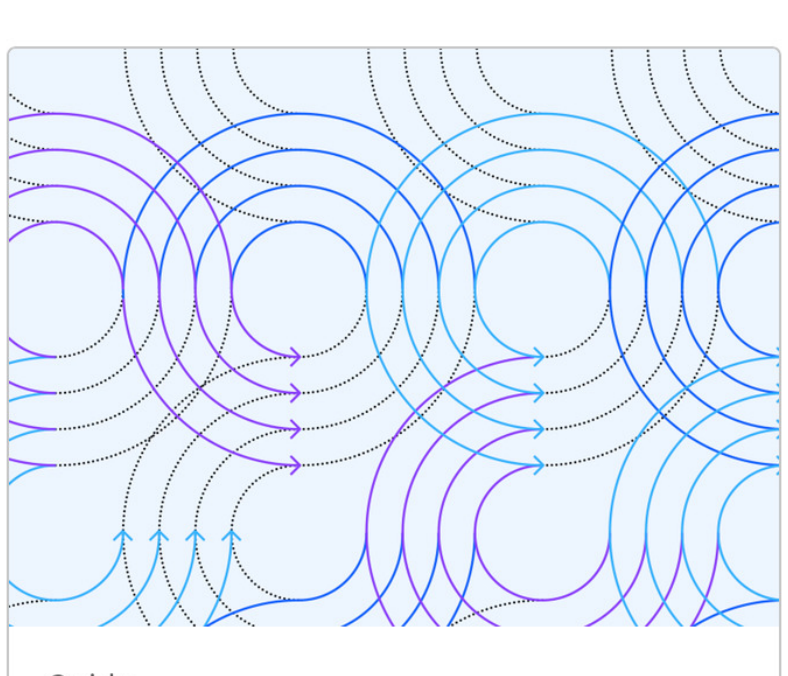
Resources



IBM Bob
Meet Bob™, your AI development partner

An AI powered tool that helps you code, debug and deliver high quality software without breaking your flow.

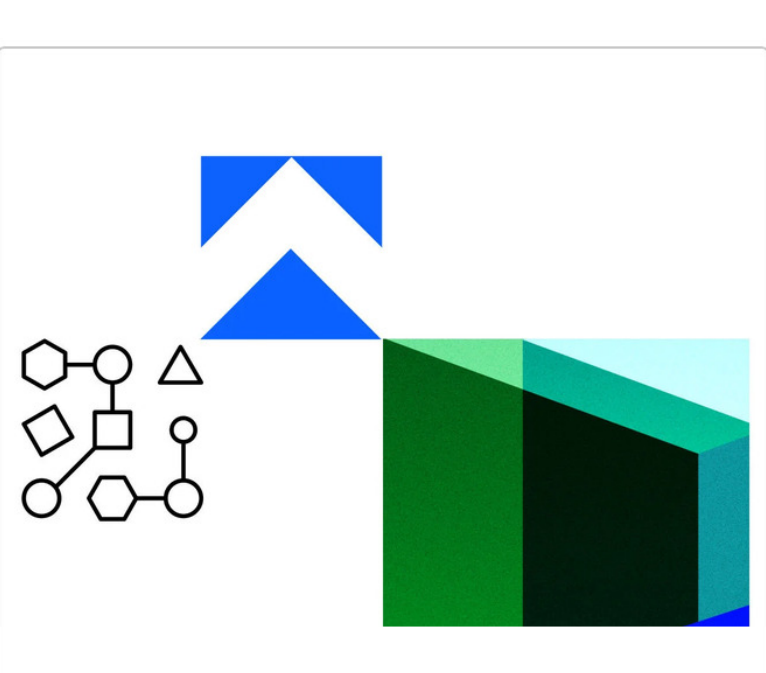
Start building with Bob →



Guide
Start realizing ROI: A practical guide to agentic AI

Learn how to scale agentic AI for measurable ROI across your enterprise. This playbook outlines the top barriers that limit impact, how to effectively measure ROI and a practical framework to drive successful, enterprise-wide adoption.

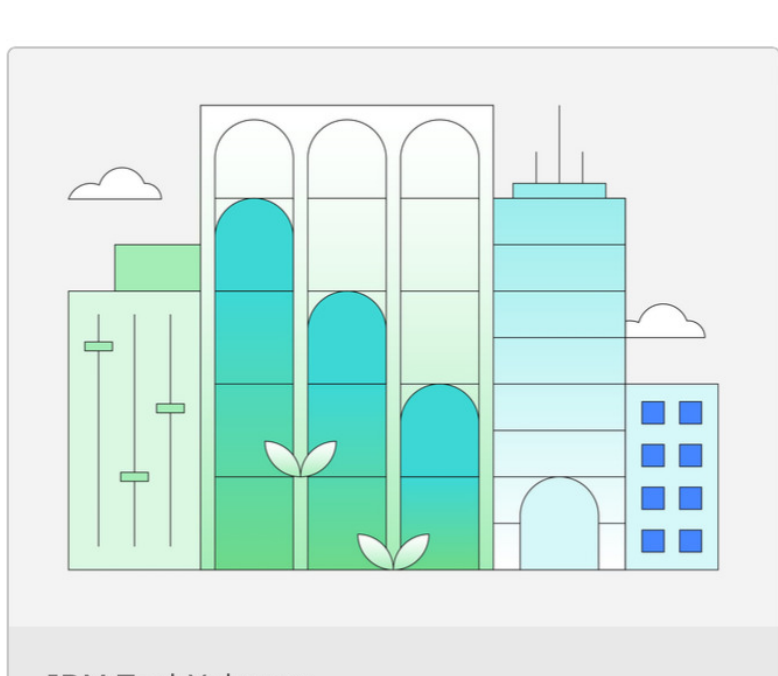
Get the guide →



AI models
Explore IBM Granite

Discover IBM Granite®, our family of open, performant and trusted AI models, tailored for business and optimized to scale your AI applications. Explore language, code, time series and guardrail options.

Learn more →



IBM TechXchange
IBM AI community

Join the community for Al architects and builders to learn, share ideas and connect with others.

Find your community →

1 / 2

Related solutions

IBM watsonx Orchestrate®
Easily design scalable AI assistants and agents, automate repetitive tasks and simplify complex processes with IBM watsonx Orchestrate.

Explore watsonx Orchestrate →

IBM Bob
Accelerate software delivery with Bob, your AI partner for secure, intent-aware development.

Explore IBM Bob →

Artificial intelligence solutions
Put AI to work in your business with IBM’s industry-leading AI expertise and portfolio of solutions at your side.

Explore AI solutions →

Take the next step

Discover how you can scale and govern your AI with watsonx Orchestrate, the control plane that helps you streamline operations, automate workflows, and manage intelligent agents with ease.

Discover watsonx Orchestrate → Explore IBM Bob →

Footnotes

[1] "Lost in the Middle: How Language Models Use Long Contexts," arXiv, 6 July 2023

[2] "Many-shot jailbreaking," Anthropic, 2 April 2024

IBM
Discover
Products
Consulting services
Industries
Case studies
Financing
Research
Connect
Business partners
Documentation
Events
Newsletters
Support
TechXchange community
About
Overview
Careers
Investor relations
Leadership
Newsroom
Security, privacy and trust
Follow
LinkedIn
X
Instagram
YouTube
Podcasts
Contact IBM
Privacy
Terms of use
Accessibility